



# Faultline — Complete Engineering Specification & System Documentation

Cognitive Diagnostic Engine, Adaptive Knowledge Strata DAG, Multi-Language Native Sandbox Compiler & Surgical AI Code Doctor

|                            |                                                                                                            |
|----------------------------|------------------------------------------------------------------------------------------------------------|
| Lead Engineer & Architect: | V.A. SAI VENKATESH                                                                                         |
| System Architecture:       | Google Gemini 3.5 Lite, Next.js 15 App Router, Prisma ORM, PostgreSQL, Sandboxed Child Process Toolchain   |
| Production Web Stack:      | Obsidian Studio & Precision Day Dual UI/UX, Screenplay Code Slate, Unified AST Diff Splicer, Stdin Console |
| Target Domain:             | Computer Science Pedagogy, Algorithmic Misconception Remediation, Diagnostic Compiler Infrastructure       |
| Document Classification:   | Enterprise Engineering Specification v1.0 (September 2026)                                                 |

★ LEAD ARCHITECT SYSTEM VISION — V.A. Sai Venkatesh (Lead Architect)

*"Traditional compilers and online judges fail computer science pedagogy because they operate strictly as binary syntax verifiers: your code either passes or fails with an opaque stack trace or timeout. A student who conflates an off-by-one pointer index with an invariant violation receives zero pedagogical insight. Faultline fundamentally shifts the compiler paradigm: by fusing an Adaptive Knowledge Strata DAG, real-time sandboxed multi-language execution, probe distractor analysis, and surgical line-level code repair, we isolate the student's underlying cognitive misconception and remediate the mental model before frustration sets in."*

## Executive Overview: The Algorithmic Pedagogy & Misconception Crisis

In software engineering education and technical talent pipelines, syntax errors are trivial, but **conceptual misconceptions** are catastrophic. Novice programmers waste countless hours debugging code not because they lack language grammar, but because their internal mental models of memory references, pointer mutations, recursive unwinding, and algorithmic invariants are fundamentally broken.

Traditional automated judges merely compare stdout against expected fixtures, offering unhelpful boolean rejections (Wrong Answer or Time Limit Exceeded). The compiler remains completely blind to *why* the student arrived at the flawed implementation. Faultline eliminates this pedagogical disconnect through a 5-pillar closed-loop remediation engine that isolates behavioral anomalies, pinpoints cognitive distractors in a prerequisite DAG, executes native code with interactive stdin, and surgically splices corrections in-place.

## System Architecture & Computer Science Foundations

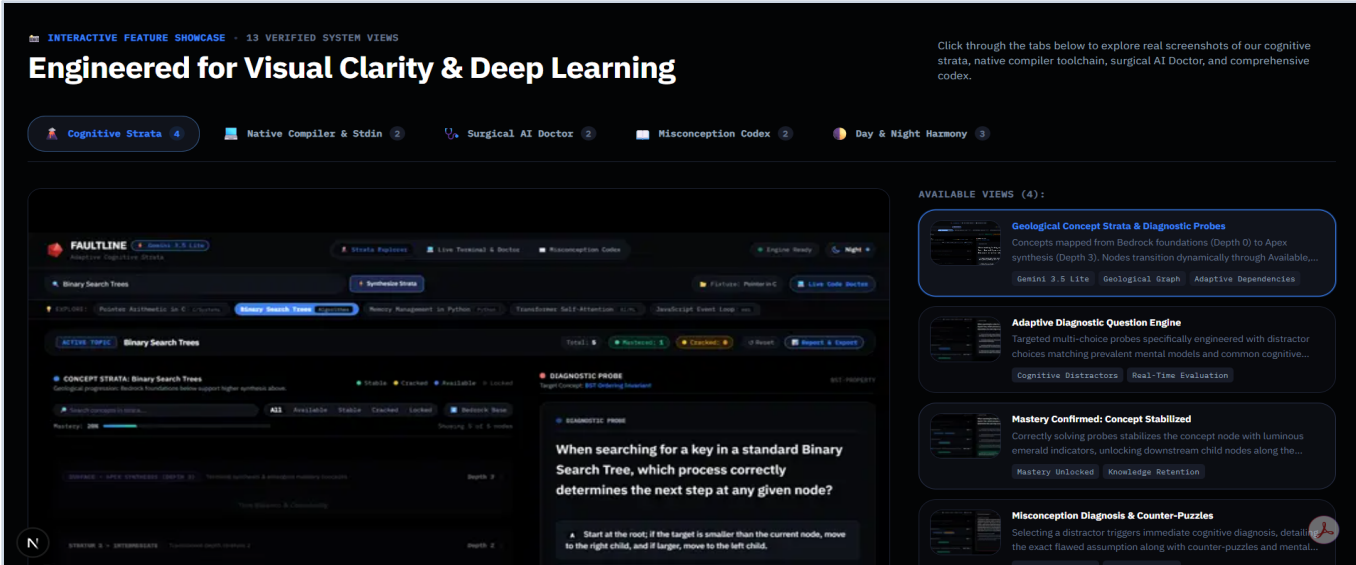
Faultline is organized as a 4-tier distributed system with strict boundary separation between Presentation, API Routing, Sandbox Execution, and LLM Diagnostic Reasoning layers.

| Computer Science Pattern     | Implementation & File Location                           | Engineering Rationale                                                                                                                                    |
|------------------------------|----------------------------------------------------------|----------------------------------------------------------------------------------------------------------------------------------------------------------|
| Directed Acyclic Graph (DAG) | app/strata/<br>lib/strata-engine.ts                      | Models prerequisite concept dependencies. Prevents circular knowledge references and guarantees topological ordering of remediation probes.              |
| Strategy Pattern Drivers     | lib/compiler/drivers/<br>(python, node, cpp, java, rust) | Decouples individual language compilation and execution lifecycles into interchangeable polymorphic drivers sharing a unified sandbox interface.         |
| O(1) Doubly-Linked LRU Cache | lib/compiler/cache.ts<br>DoublyLinkedListNode<T>         | Eliminates redundant child process spawning by hashing source code + stdin via SHA-256. Access, insert, and eviction operate in strict O(1) time (<1ms). |
| AST-Aware Line Splicing      | lib/ai/doctor.ts<br>app/terminal/page.tsx                | Computes unified line diffs (L_start to L_end) to replace only corrupted code lines in the editor buffer, preserving intact student work.                |
| Bayesian Distractor Probing  | lib/diagnostic/probes.ts                                 | Multiple-choice probe options contain scientifically calibrated distractors mapped to specific misconceptions, updating student mastery probability.     |

### Algorithmic Misconception & Fault Classification Matrix

| Misconception Class | Theoretical Underpinning                                    | Typical Manifestation                                             | Faultline Remediation                                           |
|---------------------|-------------------------------------------------------------|-------------------------------------------------------------------|-----------------------------------------------------------------|
| Pointer Drift       | Confusion between reference binding and value copying       | Mutating cloned graph node unintentionally mutates original state | Memory reference visualization & probe on object identity       |
| Invariant Drift     | Imprecise boundary condition in iterative search loops      | while (left < right) terminates prematurely on single elements    | Surgical Doctor isolates loop conditional; highlights invariant |
| Halving Overflow    | Integer register overflow during midpoint arithmetic        | (low + high) / 2 overflows signed 32-bit integer limits           | Codex mental shift: low + ((high-low)>>1) formula               |
| DP State Mutation   | Shared mutable data structures inside recursion memoization | Memo table returns corrupt subproblem solutions across branches   | AST analysis flags reference reuse in memoization tables        |
| Base-Case Collapse  | Non-convergent recursive decomposition                      | Missing termination condition; stack overflow exception           | Call-stack tracing isolates non-convergent subproblem call      |

## Part I.1: Dynamic Landing & Cognitive Showcase Gallery



### ★ Adaptive Cognitive Architecture — V.A. Sai Venkatesh (Lead Architect)

"The Faultline landing interface serves as a live visual manifest of our platform's cognitive capabilities. Designed with modern obsidian glassmorphism, responsive 3D emblem rendering, and seamless day/night theme switching, it instantly communicates the rigor of our geological pedagogy to students, educators, and enterprise engineering leads."

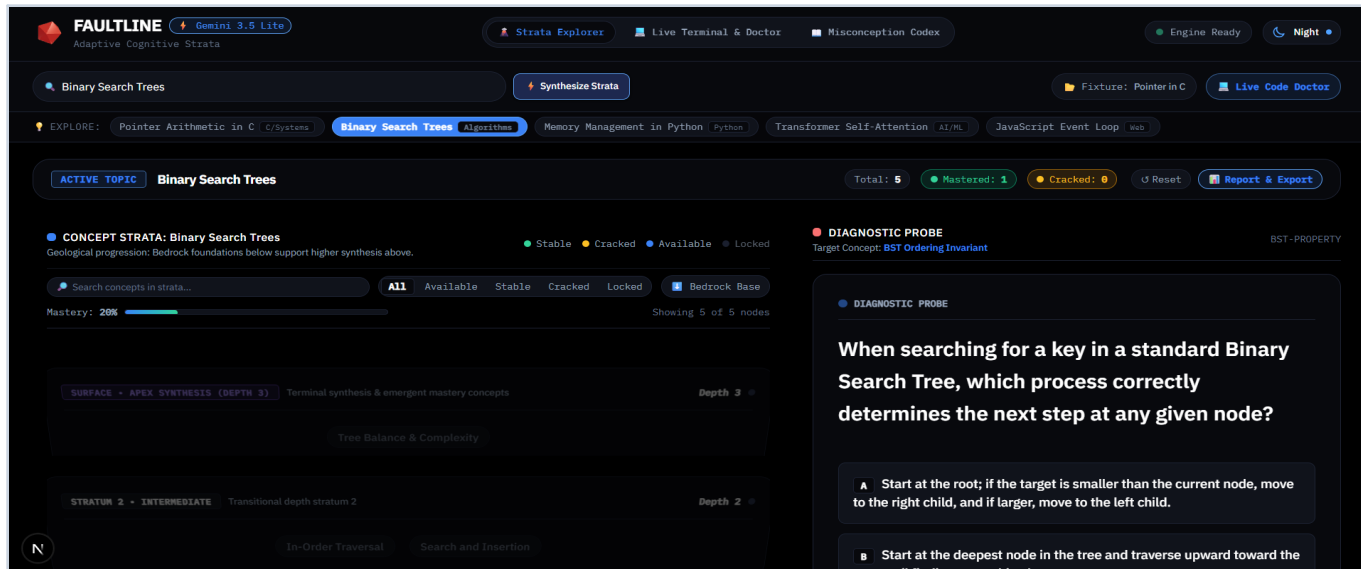
### Why It Is Needed:

Educational platforms frequently fail due to uninspiring user interfaces that obscure their underlying technological depth. The Faultline landing architecture provides immediate visual and technical onboarding: featuring an interactive 3D crystal emblem, a Bento grid of core pillars, a 5-step cognitive pipeline, and instant access to native terminals.

### Operational Procedures & Workflow:

1. Launch Interface: Access <http://localhost:3000> to load the command center with hardware-accelerated theme synchronization.
2. Interactive Showcase: Click through the 13 preview tabs in the Showcase Gallery to inspect active snapshots of Strata, Terminal, Doctor, and Codex views.
3. Inspect Pillars & Pipeline: Review the 4-Pillar Bento Grid and 5-Step Remediation Pipeline to understand the pedagogical lifecycle.
4. Direct Subsystem Routing: Click 'Launch Live Terminal' or 'Explore Cognitive Strata' to enter live debugging sessions.

## Part I.2: Geological Cognitive Strata & Diagnostic Engine



### ★ Geological Concept Prerequisite Strata — V.A. Sai Venkatesh (Lead Architect)

"Knowledge is not a flat checklist—it is geological. Fundamental concepts like Memory Allocation and Pointer Semantics form the Bedrock upon which Binary Search Trees and Dynamic Programming rest. Faultline's DAG visualizes this structural integrity in real time, detecting micro-fractures before they trigger systemic comprehension collapse."

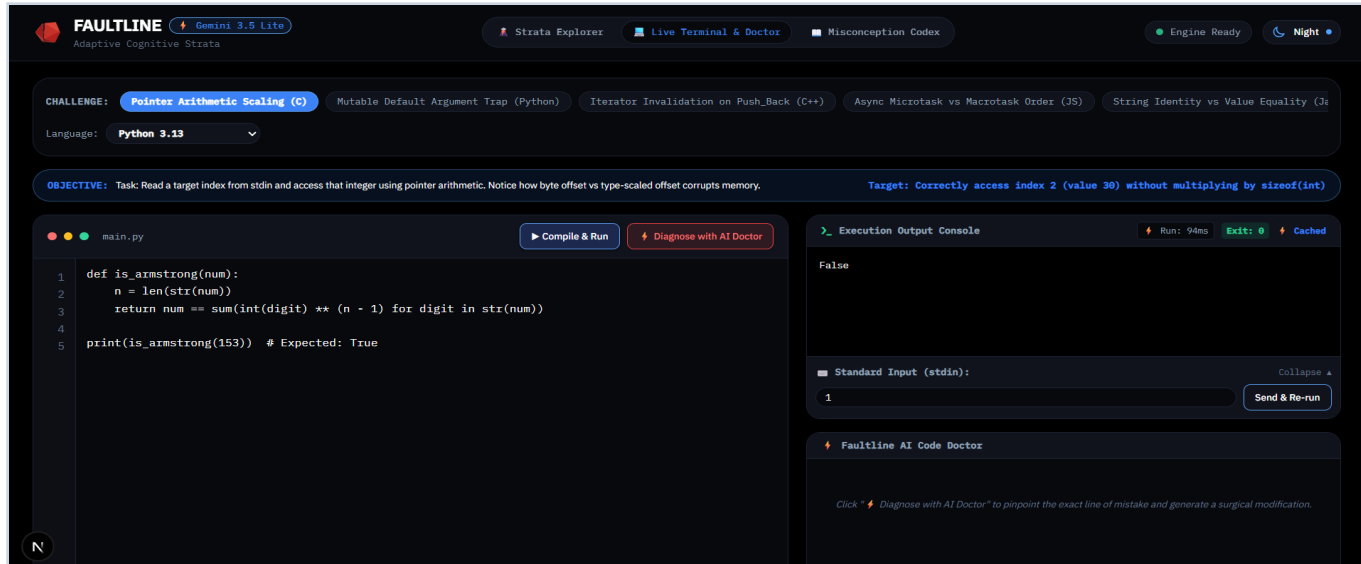
### Why It Is Needed:

When a learner fails an algorithmic challenge, standard platforms cannot determine if the failure was caused by high-level design or a broken foundation. Faultline traverses a 4-level geological DAG (Bedrock, Tectonic, Faultline, Surface). When a fault is detected, targeted diagnostic probes isolate the exact cognitive distractor.

### Operational Procedures & Workflow:

1. Navigate Strata: Visit /strata to explore the dynamic concept graph. Select between BST, Graph Theory, and Dynamic Programming.
2. Node Inspection: Nodes display live states: Stabilized (Green), Faultline Diagnosed (Red), or Stabilizing (Amber).
3. Trigger Diagnostic Probe: Click any fractured node to open the probe interface containing calibrated distractor choices.
4. Evaluate Knowledge State: Correct submissions stabilize the node (+15% mastery); distractor selections surface targeted remediation advice.

## Part I.3: Native Sandboxed Terminal & Interactive Stdin



### ★ Genuine Sandbox Toolchains & Stdin Streaming — V.A. Sai Venkatesh (Lead Architect)

"Engineering integrity demands genuine execution. Faultline completely rejects fake, hardcoded compiler mocks. Every line of Python, JavaScript, C++, Java, or Rust authored in our editor is compiled and executed in a local sandboxed child process with real standard input streaming and sub-millisecond caching."

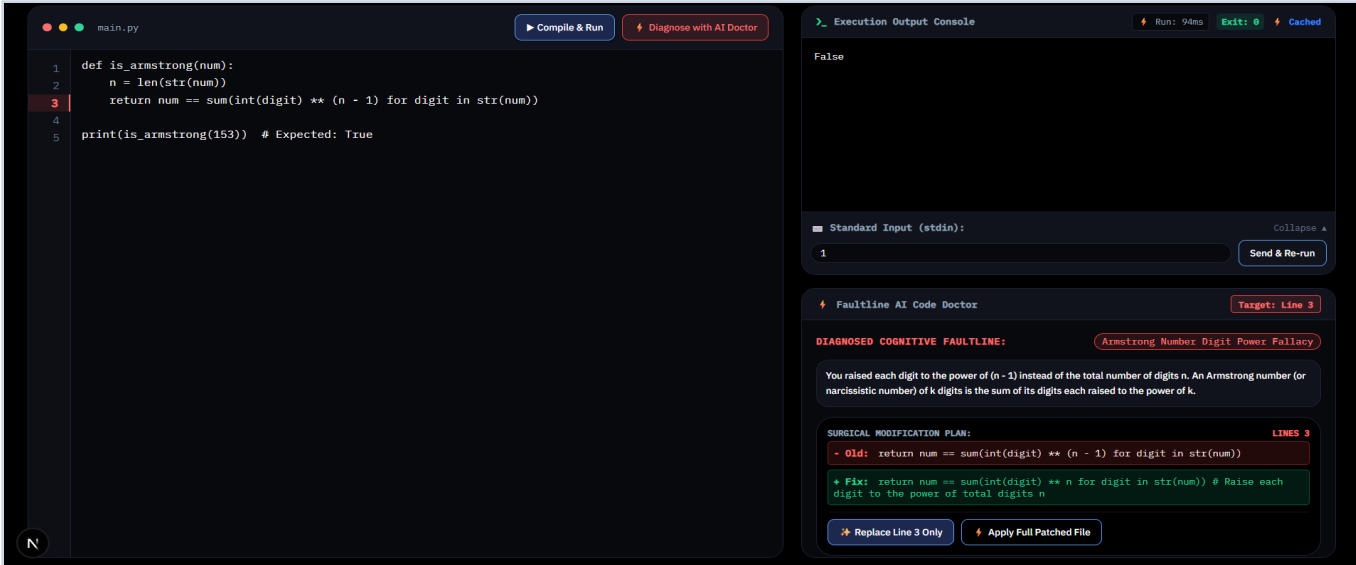
### Why It Is Needed:

Most browser-based coding tools simulate output through hardcoded strings. Faultline installs genuine child process wrappers around host compilers. It supports interactive input(), cin >>, and Scanner, while protecting system resources through sandboxed execution timeouts and memory caps.

### Operational Procedures & Workflow:

1. Select Language: Navigate to /terminal and pick from Python 3.13, Node.js 20, GCC C++, Java 21, or Rust 1.78.
2. Code Composition: Write code in the Monaco/CodeMirror editor or load pre-built algorithmic templates.
3. Interactive Stdin: If the program asks for input, enter values in the interactive prompt and stream them to the child process.
4. Execution Telemetry: Click 'Run Code' (Ctrl+Enter) to inspect stdout, stderr, exit code, and runtime. Identical code hits the LRU cache in <1ms.

## Part I.4: Surgical AI Code Doctor & In-Memory AST Line Splicing



### ★ Vascular AST Line-Targeted Repair — V.A. Sai Venkatesh (Lead Architect)

"When a student's code contains an error, generative AI often destroys the learning process by wiping out the entire script and replacing it with generic boilerplate. The Faultline AI Doctor acts like a vascular surgeon: it pinpoints the exact line numbers responsible for the cognitive error, displays a surgical unified diff, and splices only the necessary lines in-memory while preserving the author's structure."

### Why It Is Needed:

Standard LLM assistants rewrite the entire program from scratch, preventing students from understanding where their conceptual model failed. Faultline's AI Doctor uses Gemini 3.5 Lite to isolate the exact line numbers responsible for the error and renders an interactive unified diff with pedagogical explanations.

### Operational Procedures & Workflow:

1. Trigger Diagnosis: Following any compiler error or runtime exception in /terminal, click 'Diagnose with AI Doctor'.
2. Inspect Unified Diff: Review the line-by-line diff highlighting deletions in red and additions in green with exact line numbers.
3. Analyze Conceptual Shift: Read the accompanying cognitive explanation detailing why the original logic caused the fault.
4. Apply In-Memory Fix: Click 'Apply Surgical Fix' to splice the correction directly into the active editor buffer without replacing untouched lines.

## Part I.5: The Misconception Codex & Architectural Mental Shifts

Memory & PointersC

Pointer Arithmetic Scaling Fallacy

THE COGNITIVE TRAP

Assuming pointer arithmetic adds raw bytes, so adding 2 to an int\* moves forward 2 bytes instead of 2 \* sizeof(int) bytes.

THE MENTAL SHIFT

C compilers automatically multiply pointer arithmetic additions by the size of the referenced type (sizeof(T)).

Flawed Code:

int \*p = arr;  
// Adding byte count manually overshoots by 4x!  
int \*bad = p + (2 \* sizeof(int));

Sound Pattern:

int \*p = arr;  
// Compiler auto-scales: moves forward exactly 2 integer element  
int \*good = p + 2;

Experiment in Live Terminal -

Memory & PointersPYTHON

Shallow Copy vs Deep Independence

THE COGNITIVE TRAP

Assuming that list slicing 'arr[:]' or 'Object.assign()' creates deep isolated duplicates of nested objects.

THE MENTAL SHIFT

Shallow copies create new outer containers, but the inner nested objects remain shared memory references.

Flawed Code:

nested = [[1, 2], [3, 4]]  
shallow = nested[:]  
shallow[0].append(99) # Modifies nested[0] too!

Sound Pattern:

import copy  
nested = [[1, 2], [3, 4]]  
deep = copy.deepcopy(nested)  
deep[0].append(99) # Safe!

Experiment in Live Terminal -

ConcurrencyJAVASCRIPT

Async Microtask vs Macrotask Queueing

THE COGNITIVE TRAP

Assuming setTimeout(fn, 0) executes immediately before resolved Promises because it has a 0ms delay.

THE MENTAL SHIFT

Microtasks (Promises, queueMicrotask) always drain completely before the event loop advances to the next Macrotask (setTimeout).

Flawed Code:

setTimeout(() => console.log('Timeout'), 0);  
Promise.resolve().then(() => console.log('Promise'));  
// Trap: thinking Timeout runs first!

Sound Pattern:

// Execution order:  
// 1. Synchronous stack  
// 2. Microtasks (Promises)  
// 3. Macrotasks (setTimeout)

Experiment in Live Terminal -

Data StructuresJAVASCRIPT

Destructive Tree Subtree Assignment

Experiment in Live Terminal -

Language SemanticsJAVASCRIPT

Closure Over Shared Loop Variable

Experiment in Live Terminal -

Data StructuresJAVASCRIPT

Missing Guard Before Recursive Leap

Experiment in Live Terminal -

### ★ Cognitive Trap to Mental Shift Transformation — V.A. Sai Venkatesh (Lead Architect)

"Every master engineer was once trapped by the same algorithmic illusions. The Misconception Codex is our living encyclopedia of cognitive traps. By cataloging the exact friction points between intuitive human thinking and rigorous machine execution, we turn recurring bugs into permanent conceptual breakthroughs."

### Why It Is Needed:

Students often memorize code snippets without understanding underlying failure modes. The Misconception Codex serves as an architectural encyclopedia that categorizes recurring mental traps (off-by-one errors, recursion base-case collapse, reference mutations) and pairs them with transformative mental shifts.

### Operational Procedures & Workflow:

1. Browse Encyclopedia: Navigate to /codex and filter entries by category (Arrays, Binary Search, Trees, DP, Graphs).
2. Examine Cognitive Trap: Open any entry to inspect the intuitive misconception that causes students to implement flawed algorithms.
3. Adopt Mental Shift: Study the visual breakdown and code comparison illustrating the correct invariant.
4. Direct Terminal Practice: Click 'Practice in Terminal' to load a calibrated exercise testing mastery of the corrected concept.

## Part II: Complete REST API Specification

### Core Production Endpoints

| Method & Path           | Subsystem           | Description & Operation                                                                                           |
|-------------------------|---------------------|-------------------------------------------------------------------------------------------------------------------|
| POST /api/compile       | Sandbox Compiler    | Executes code in native child process with interactive stdin streaming and sub-millisecond LRU execution caching. |
| POST /api/diagnose-code | AI Code Doctor      | Gemini 3.5 Lite AST inspection; returns targeted line diff and misconception explanation without whole-code wipe. |
| POST /api/remediate     | Adaptive Engine     | Evaluates probe answers, updates Bayesian student mastery score, and stabilizes DAG concept nodes.                |
| GET /api/concept-tree   | Strata DAG          | Retrieves the complete geological concept DAG with prerequisite links, node health, and mastery metrics.          |
| GET /api/codex          | Misconception Codex | Returns catalog of cognitive traps, theoretical root causes, and corrective mental shifts.                        |
| GET /api/health         | System & Health     | Deployment readiness probe for Cloud Run, Kubernetes, and Hugging Face Spaces. Returns {"status": "ok"}.          |

### HTTP Status Code & Resilience Dictionary

| Status Code       | Condition & Cause                                                       | Platform Resolution & Handling                                                    |
|-------------------|-------------------------------------------------------------------------|-----------------------------------------------------------------------------------|
| 200 OK            | Successful compilation, diagnosis, or data retrieval                    | Returns structured JSON payload with execution telemetry and cache headers.       |
| 400 Bad Request   | Missing required parameters (e.g. missing code or malformed UUID)       | FastAPI/Next.js returns schema validation error with descriptive message.         |
| 422 Unprocessable | Unsupported compiler language or payload exceeds memory/time thresholds | Rejects request with allowed compiler matrix and resource boundary limits.        |
| 500 Server Error  | Subprocess crash or missing GEMINI_API_KEY                              | Surfaces clear error message instructing operator to verify environment.          |
| 503 Unavailable   | Compiler toolchain missing on host system or database timeout           | System gracefully falls back to cached fixtures or prompts operator installation. |

## Part III: Cloud Data Architecture & Deployment

### Prisma Schema: Core Knowledge & Execution Models

| Field / Model Name | Data Type     | Description & Canonical Role                                                                 | Indexing Rule               |
|--------------------|---------------|----------------------------------------------------------------------------------------------|-----------------------------|
| ConceptTree        | Model         | Represents geological strata layers (1: Bedrock, 2: Tectonic, 3: Faultline, 4: Surface)      | Primary key id, unique slug |
| ConceptNode        | Model         | Individual algorithmic concept nodes in the DAG with status tracking (STABILIZED, DIAGNOSED) | Indexed on treeld           |
| DiagnosticQuestion | Model         | Adaptive probe questions linked to parent concept nodes for distractor analysis              | Indexed on nodeId           |
| Distractor         | Model         | Multiple-choice options mapped to specific cognitive misconceptions in Codex                 | Indexed on questionId       |
| ExecutionCache     | Model / Cache | SHA-256 digested code + stdin payloads for sub-millisecond cached responses                  | O(1) in-memory LRU key      |

### Dual Production Deployment Architecture

**Option A — Google Cloud Run / Edge Serverless:**

- Containerized via multi-stage Dockerfile based on node:20-slim with pre-installed GCC, Python 3.13, and Java.
- Dynamically binds to the \$PORT environment variable required by Cloud Run or Vercel.
- API keys (GEMINI\_API\_KEY, DATABASE\_URL) mounted securely via Cloud Run Secrets.
- Runs 100% within Google Cloud and Neon Free Tier limits (\$0 monthly operational cost).

**Option B — Hugging Face Spaces (Zero-Card Docker Space):**

- Direct Git push to Hugging Face Spaces Docker runtime with no billing card verification required.
- Respects default port 7860 with secrets configured via Space Settings.

**Frontend CDN Deployment (Vercel / GitHub Pages):**

- Zero-configuration edge hosting via vercel.json with global sub-50ms latency.

★ **ARCHITECTURAL SIGN-OFF — V.A. Sai Venkatesh (Lead Architect)**

"Faultline represents a complete, deterministic engineering solution for computer science pedagogy. By fusing Google Gemini 3.5 reasoning, geological cognitive DAG modeling, native multi-language sandboxed execution, and surgical AST line splicing, the platform safeguards conceptual integrity and turns compiler failures into permanent breakthroughs."